SR University **Name: G. Akhila**

Subject: AI Assisted coding **2403A51L40**

Assignment 12.5

Task 1: Sorting – Merge Sort Implementation

Prompt: Generate a Python program that implements the Merge Sort algorithm. Create a function merge_sort(arr) that takes an **unsorted list**, sorts it in ascending order, and prints the sorted list. Include a docstring explaining time and space complexity, and verify the code using test cases.

```python
def merge_sort(arr):
    """
    Sorts an array in ascending order using the Merge Sort algorithm.
    Time Complexity: O(n log n) - The array is divided into halves log n times, and each merge operation takes O(n) time.
    Space Complexity: O(n) - Additional space is required for the temporary arrays used during the merge process.
    Parameters:
    arr (list): The unsorted list to be sorted.
    Returns:
    None: The function prints the sorted list.
    """
    if len(arr) > 1:
        mid = len(arr) // 2  # Finding the mid of the array
        L = arr[:mid]        # Dividing the elements into 2 halves
        R = arr[mid:]
        merge_sort(L)        # Sorting the first half
        merge_sort(R)        # Sorting the second half
        i = j = k = 0
        # Copy data to temp arrays L[] and R[]
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1
        # Checking if any element was left
        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1

        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1
# Test cases
if __name__ == "__main__":
    test_cases = [
        [38, 27, 43, 3, 9, 82, 10],
        [12, 11, 13, 5, 6, 7],
        [5, 2, 9, 1, 5, 6],
        [1],
        []
    ]
    for i, test in enumerate(test_cases):
        print(f"Test case {i + 1}: Original array: {test}")
        merge_sort(test)
        print(f"Sorted array: {test}\n")
```

Output:

```
Test case 1: Original array: [38, 27, 43, 3, 9, 82, 10]
Sorted array: [3, 9, 10, 27, 38, 43, 82]

Test case 2: Original array: [12, 11, 13, 5, 6, 7]
Sorted array: [5, 6, 7, 11, 12, 13]

Test case 3: Original array: [5, 2, 9, 1, 5, 6]
Sorted array: [1, 2, 5, 5, 6, 9]

Test case 4: Original array: [1]
Sorted array: [1]

Test case 5: Original array: []
Sorted array: []
```

Explanation:

Merge Sort is used to sort an unsorted list efficiently. It divides the list into smaller parts, sorts them, and merges them back in ascending order. It has O(n log n) time complexity and uses extra space

---

Task 2: Searching – Binary Search with AI Optimization

Prompt: Create a Python function binary_search(arr, target) that searches for a target value. The input list should be **unsorted initially**, then sorted before applying binary search. Return the index if found or -1 if not found. Include a docstring explaining best, average, and worst-case complexities and test with multiple inputs.

```python
def binary_search(arr, target):
    """
    Searches for a target value in an unsorted list by first sorting it and then applying binary search.
    Best Case Complexity: O(1) - When the target is found at the middle of the sorted array.
    Average Case Complexity: O(log n) - The array is halved with each iteration.
    Worst Case Complexity: O(log n) - When the target is not found, the search will go through log n iterations.
    Parameters:
    arr (list): The unsorted list to be searched.
    target: The value to search for.
    Returns:
    int: The index of the target if found, otherwise -1.
    """
    # First, sort the array using merge_sort
    merge_sort(arr)
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1
    # Target was not found in the array
    return -1
# Test cases
if __name__ == "__main__":
    test_cases = [
        ([38, 27, 43, 3, 9, 82, 10], 9),
        ([12, 11, 13, 5, 6, 7], 5),
        ([5, 2, 9, 1, 5, 6], 10),
        ([1], 1),
        ([], 0)
    ]
    for i, (test_arr, target) in enumerate(test_cases):
        print(f"Test case {i + 1}: Original array: {test_arr}, Target: {target}")
        index = binary_search(test_arr, target)
        if index != -1:
            print(f"Target found at index: {index}\n")
        else:
            print("Target not found in the array.\n")
```

Output:

```
Test case 1: Original array: [38, 27, 43, 3, 9, 82, 10], Target: 9
Target found at index: 1

Test case 2: Original array: [12, 11, 13, 5, 6, 7], Target: 5
Target found at index: 0

Test case 3: Original array: [5, 2, 9, 1, 5, 6], Target: 10
Target not found in the array.

Test case 4: Original array: [1], Target: 1
Target found at index: 0

Test case 5: Original array: [], Target: 0
Target not found in the array.
```

Explanation:

Binary Search is used to find an element quickly in a sorted list. The unsorted list is first sorted, then the search space is reduced by half in each step, making the search fast.

Task 3: Smart Healthcare Appointment Scheduling System

Prompt: A healthcare system stores appointment records as an **unsorted list of tuples** (appointment ID, patient name, doctor name, time, fee). Recommend suitable searching and sorting algorithms, justify the choices, and implement Python code to search by appointment ID and sort by time or consultation fee, printing the sorted output.

```python
def search_by_appointment_id(records, appointment_id):
    """
    Searches for an appointment record by appointment ID.
    Parameters:
    records (list of tuples): The unsorted list of appointment records.
    appointment_id: The appointment ID to search for.
    Returns:
    tuple: The appointment record if found, otherwise None.
    """
    for record in records:
        if record[0] == appointment_id:
            return record
    return None
def sort_by_time(records):
    """
Sorts appointment records by time.
    Parameters:
    records (list of tuples): The unsorted list of appointment records.
    Returns:
    list of tuples: The sorted list of appointment records by time.
    """
    return sorted(records, key=lambda x: x[3])
def sort_by_fee(records):
    """Sorts appointment records by consultation fee.
    Parameters:
    records (list of tuples): The unsorted list of appointment records.
    Returns:
    list of tuples: The sorted list of appointment records by fee.
    """
    return sorted(records, key=lambda x: x[4])
# Test cases
if __name__ == "__main__":
    appointment_records = [
        (101, "Alice", "Dr. Smith", "10:00 AM", 150),
        (102, "Bob", "Dr. Jones", "11:00 AM", 200),
        (103, "Charlie", "Dr. Brown", "9:00 AM", 100),
        (104, "David", "Dr. Taylor", "1:00 PM", 250)
    ]
    # Search by appointment ID
    print("Searching for appointment ID 102:")
    record = search_by_appointment_id(appointment_records, 102)
    if record:
        print(f"Record found: {record}\n")
    else:
        print("Record not found.\n")
    # Sort by time
    print("Sorting by time:")
    sorted_by_time = sort_by_time(appointment_records)
    for record in sorted_by_time:
        print(record)
    print()
    # Sort by fee
    print("Sorting by consultation fee:")
    sorted_by_fee = sort_by_fee(appointment_records)
    for record in sorted_by_fee:
        print(record)
```

Output:

```
Searching for appointment ID 102:
Record found: (102, 'Bob', 'Dr. Jones', '11:00 AM', 200)

Sorting by time:
(101, 'Alice', 'Dr. Smith', '10:00 AM', 150)
(102, 'Bob', 'Dr. Jones', '11:00 AM', 200)
(104, 'David', 'Dr. Taylor', '1:00 PM', 250)
(103, 'Charlie', 'Dr. Brown', '9:00 AM', 100)

Sorting by consultation fee:
(103, 'Charlie', 'Dr. Brown', '9:00 AM', 100)
(101, 'Alice', 'Dr. Smith', '10:00 AM', 150)
(102, 'Bob', 'Dr. Jones', '11:00 AM', 200)
(104, 'David', 'Dr. Taylor', '1:00 PM', 250)
```

Explanation:

Searching by appointment ID is done using an efficient search method, while sorting helps arrange appointments by time or fee. This improves quick access and better scheduling management.

Task 4: Railway Ticket Reservation System

Prompt: A railway reservation system stores ticket details in an **unsorted list of tuples** (ticket ID, passenger name, train number, seat number, travel date). Use AI to identify efficient searching and sorting algorithms, justify them, and implement Python code to search tickets by ID and sort bookings by travel date or seat number, printing the sorted list.

```python
def search_by_ticket_id(records, ticket_id):
    """
    Searches for a ticket record by ticket ID.
    Parameters:
    records (list of tuples): The unsorted list of ticket records.
    ticket_id: The ticket ID to search for.
    Returns:
    tuple: The ticket record if found, otherwise None.
    """
    for record in records:
        if record[0] == ticket_id:
            return record
    return None
def sort_by_travel_date(records):
    """
Sorts ticket records by travel date.
Parameters:
records (list of tuples): The unsorted list of ticket records.
Returns:
list of tuples: The sorted list of ticket records by travel date.
"""
    return sorted(records, key=lambda x: x[4])
def sort_by_seat_number(records):
    """Sorts ticket records by seat number.
    Parameters:
    records (list of tuples): The unsorted list of ticket records.
    Returns:
    list of tuples: The sorted list of ticket records by seat number.
    """
    return sorted(records, key=lambda x: x[3])
# Test cases
if __name__ == "__main__":
    ticket_records = [
        (201, "Alice", "Train A", "12A", "2023-10-01"),
        (202, "Bob", "Train B", "15B", "2023-09-15"),
        (203, "Charlie", "Train C", "10C", "2023-10-05"),
        (204, "David", "Train D", "20D", "2023-09-20")
    ]
    # Search by ticket ID
    print("Searching for ticket ID 202:")
    record = search_by_ticket_id(ticket_records, 202)
    if record:
        print(f"Record found: {record}\n")
    else:
        print("Record not found.\n")
    # Sort by travel date
    print("Sorting by travel date:")
    sorted_by_date = sort_by_travel_date(ticket_records)
    for record in sorted_by_date:
        print(record)
    print()
    # Sort by seat number
    print("Sorting by seat number:")
    sorted_by_seat = sort_by_seat_number(ticket_records)
    for record in sorted_by_seat:
        print(record)
```

Output:

```
Searching for ticket ID 202:
Record found: (202, 'Bob', 'Train B', '15B', '2023-09-15')

Sorting by travel date:
(202, 'Bob', 'Train B', '15B', '2023-09-15')
(204, 'David', 'Train D', '20D', '2023-09-20')
(201, 'Alice', 'Train A', '12A', '2023-10-01')
(203, 'Charlie', 'Train C', '10C', '2023-10-05')

Sorting by seat number:
(203, 'Charlie', 'Train C', '10C', '2023-10-05')
(201, 'Alice', 'Train A', '12A', '2023-10-01')
(202, 'Bob', 'Train B', '15B', '2023-09-15')
(204, 'David', 'Train D', '20D', '2023-09-20')
```

Explanation:

Searching helps quickly find ticket details using ticket ID. Sorting tickets by date or seat number helps in organizing bookings and generating reports easily.

Task 5: Smart Hostel Room Allocation System

Prompt: A hostel management system maintains an **unsorted list of tuples** containing student ID, room number, floor, and allocation date. Use AI to suggest optimized searching and sorting algorithms, justify the selections, and implement Python code to search by student ID and sort records by room number or allocation date, displaying the sorted output.

```python
def search_by_student_id(records, student_id):
    """
    Searches for a student record by student ID.
    Parameters:
    records (list of tuples): The unsorted list of student records.
    student_id: The student ID to search for.
    Returns:
    tuple: The student record if found, otherwise None.
    """
    for record in records:
        if record[0] == student_id:
            return record
    return None
def sort_by_room_number(records):
    """
Sorts student records by room number.
Parameters:
records (list of tuples): The unsorted list of student records.
Returns:
list of tuples: The sorted list of student records by room number.
"""
    return sorted(records, key=lambda x: x[1])
def sort_by_allocation_date(records):
    """Sorts student records by allocation date.
    Parameters:
    records (list of tuples): The unsorted list of student records.
    Returns:
    list of tuples: The sorted list of student records by allocation date.
    """
    return sorted(records, key=lambda x: x[3])
# Test cases
if __name__ == "__main__":
    student_records = [
        (301, "Alice", "101", "2023-09-01"),
        (302, "Bob", "102", "2023-09-15"),
        (303, "Charlie", "101", "2023-10-01"),
        (304, "David", "103", "2023-09-20")
    ]
    # Search by student ID
    print("Searching for student ID 302:")
    record = search_by_student_id(student_records, 302)
    if record:
        print(f"Record found: {record}\n")
    else:
        print("Record not found.\n")
    # Sort by room number
    print("Sorting by room number:")
    sorted_by_room = sort_by_room_number(student_records)
    for record in sorted_by_room:
        print(record)
    print()
    # Sort by allocation date
    print("Sorting by allocation date:")
    sorted_by_date = sort_by_allocation_date(student_records)
    for record in sorted_by_date:
        print(record)
```

Output:

```
Searching for student ID 302:
Record found: (302, 'Bob', '102', '2023-09-15')

Sorting by room number:
(301, 'Alice', '101', '2023-09-01')
(302, 'Bob', '102', '2023-09-15')
(303, 'Charlie', '101', '2023-10-01')
(304, 'David', '103', '2023-09-20')

Sorting by allocation date:
(301, 'Alice', '101', '2023-09-01')
(302, 'Bob', '102', '2023-09-15')
(304, 'David', '103', '2023-09-20')
(303, 'Charlie', '101', '2023-10-01')
```

Explanation:

Searching by student ID allows fast access to allocation details. Sorting by room number or date helps manage hostel records in an organized manner.

Task 6: Online Movie Streaming Platform

Prompt: A movie streaming platform stores movie records as an **unsorted list of tuples** (movie ID, title, genre, rating, release year). Recommend appropriate searching and sorting algorithms, justify the choices, and implement Python functions to search by movie ID and sort movies by rating or release year, printing the sorted list.

```python
def search_by_movie_id(records, movie_id):
    """
    Searches for a movie record by movie ID.
    Parameters:
    records (list of tuples): The unsorted list of movie records.
    movie_id: The movie ID to search for.
    Returns:
    tuple: The movie record if found, otherwise None.
    """
    for record in records:
        if record[0] == movie_id:
            return record
    return None
def sort_by_rating(records):
    """
Sorts movie records by rating.
Parameters:
records (list of tuples): The unsorted list of movie records.
Returns:
list of tuples: The sorted list of movie records by rating.
"""
    return sorted(records, key=lambda x: x[3], reverse=True)
def sort_by_release_year(records):
    """Sorts movie records by release year.
    Parameters:
    records (list of tuples): The unsorted list of movie records.
    Returns:
    list of tuples: The sorted list of movie records by release year.
    """
    return sorted(records, key=lambda x: x[4])
# Test cases
if __name__ == "__main__":
    movie_records = [
        (401, "Inception", "Sci-Fi", 8.8, 2010),
        (402, "The Godfather", "Crime", 9.2, 1972),
        (403, "The Dark Knight", "Action", 9.0, 2008),
        (404, "Pulp Fiction", "Crime", 8.9, 1994)
    ]
    # Search by movie ID
    print("Searching for movie ID 402:")
    record = search_by_movie_id(movie_records, 402)
    if record:
        print(f"Record found: {record}\n")
    else:
        print("Record not found.\n")
    # Sort by rating
    print("Sorting by rating:")
    sorted_by_rating = sort_by_rating(movie_records)
    for record in sorted_by_rating:
        print(record)
    print()
    # Sort by release year
    print("Sorting by release year:")
    sorted_by_year = sort_by_release_year(movie_records)
    for record in sorted_by_year:
        print(record)
```

Output:

```
Searching for movie ID 402:
Record found: (402, 'The Godfather', 'Crime', 9.2, 1972)

Sorting by rating:
(402, 'The Godfather', 'Crime', 9.2, 1972)
(403, 'The Dark Knight', 'Action', 9.0, 2008)
(404, 'Pulp Fiction', 'Crime', 8.9, 1994)
(401, 'Inception', 'Sci-Fi', 8.8, 2010)

Sorting by release year:
(402, 'The Godfather', 'Crime', 9.2, 1972)
(404, 'Pulp Fiction', 'Crime', 8.9, 1994)
(403, 'The Dark Knight', 'Action', 9.0, 2008)
(401, 'Inception', 'Sci-Fi', 8.8, 2010)
```

Explanation:
Searching is used to quickly find movies by ID. Sorting movies by rating or release year helps users view top-rated or latest movies easily.

Task 7: Smart Agriculture Crop Monitoring System

Prompt: An agriculture monitoring system maintains crop data in an **unsorted list of tuples** (crop ID, crop name, moisture level, temperature, yield estimate). Use AI-assisted reasoning to select suitable searching and sorting algorithms, justify their suitability, and implement Python code to search by crop ID and sort by moisture level or yield estimate, printing the sorted output.

```python
def search_by_crop_id(records, crop_id):
    """
    Searches for a crop record by crop ID.
    Parameters:
    records (list of tuples): The unsorted list of crop records.
    crop_id: The crop ID to search for.
    Returns:
    tuple: The crop record if found, otherwise None.
    """
    for record in records:
        if record[0] == crop_id:
            return record
    return None
def sort_by_moisture_level(records):
    """
Sorts crop records by moisture level.
Parameters:
records (list of tuples): The unsorted list of crop records.
Returns:
list of tuples: The sorted list of crop records by moisture level.
"""
    return sorted(records, key=lambda x: x[2])
def sort_by_yield_estimate(records):
    """Sorts crop records by yield estimate.
    Parameters:
    records (list of tuples): The unsorted list of crop records.
    Returns:
    list of tuples: The sorted list of crop records by yield estimate.
    """
    return sorted(records, key=lambda x: x[4], reverse=True)
# Test cases
if __name__ == "__main__":
    crop_records = [
        (501, "Wheat", 30, 25, 1000),
        (502, "Corn", 40, 30, 1500),
        (503, "Rice", 35, 28, 1200),
        (504, "Barley", 25, 22, 800)
    ]
    # Search by crop ID
    print("Searching for crop ID 502:")
    record = search_by_crop_id(crop_records, 502)
    if record:
        print(f"Record found: {record}\n")
    else:
        print("Record not found.\n")

    # Sort by moisture level
    print("Sorting by moisture level:")
    sorted_by_moisture = sort_by_moisture_level(crop_records)
    for record in sorted_by_moisture:
        print(record)
    print()
    # Sort by yield estimate
    print("Sorting by yield estimate:")
    sorted_by_yield = sort_by_yield_estimate(crop_records)
    for record in sorted_by_yield:
        print(record)
```

Output:

```
Searching for crop ID 502:
Record found: (502, 'Corn', 40, 30, 1500)

Sorting by moisture level:
(504, 'Barley', 25, 22, 800)
(501, 'Wheat', 30, 25, 1000)
(503, 'Rice', 35, 28, 1200)
(502, 'Corn', 40, 30, 1500)

Sorting by yield estimate:
(502, 'Corn', 40, 30, 1500)
(503, 'Rice', 35, 28, 1200)
(501, 'Wheat', 30, 25, 1000)
(504, 'Barley', 25, 22, 800)
```

Explanation:
Searching helps farmers quickly access crop details. Sorting crops by moisture level or yield helps in monitoring and decision-making.

Task 8: Airport Flight Management System

Prompt: An airport flight management system stores flight information as an **unsorted list of tuples** (flight ID, airline name, departure time, arrival time, status). Use AI to recommend efficient searching and sorting algorithms, justify the selection, and implement Python code to search flight details by ID and sort flights by departure or arrival time, displaying the sorted list.

```python
def search_by_flight_id(records, flight_id):
    """
    Searches for a flight record by flight ID.
    Parameters:
    records (list of tuples): The unsorted list of flight records.
    flight_id: The flight ID to search for.
    Returns:
    tuple: The flight record if found, otherwise None.
    """
    for record in records:
        if record[0] == flight_id:
            return record
    return None
def sort_by_departure_time(records):
    """
Sorts flight records by departure time.
Parameters:
records (list of tuples): The unsorted list of flight records.
Returns:
list of tuples: The sorted list of flight records by departure time.
"""
    return sorted(records, key=lambda x: x[2])
def sort_by_arrival_time(records):
    """Sorts flight records by arrival time.
    Parameters:
    records (list of tuples): The unsorted list of flight records.
    Returns:
    list of tuples: The sorted list of flight records by arrival time.
    """
    return sorted(records, key=lambda x: x[3])
# Test cases
if __name__ == "__main__":
    flight_records = [
        (601, "Airline A", "10:00 AM", "12:00 PM", "On Time"),
        (602, "Airline B", "11:00 AM", "1:00 PM", "Delayed"),
        (603, "Airline C", "9:00 AM", "11:00 AM", "On Time"),
        (604, "Airline D", "1:00 PM", "3:00 PM", "Cancelled")
    ]
    # Search by flight ID
    print("Searching for flight ID 602:")
    record = search_by_flight_id(flight_records, 602)
    if record:
        print(f"Record found: {record}\n")
    else:
        print("Record not found.\n")
    # Sort by departure time
    print("Sorting by departure time:")
    sorted_by_departure = sort_by_departure_time(flight_records)
    for record in sorted_by_departure:
        print(record)
    print()
    # Sort by arrival time
    print("Sorting by arrival time:")
    sorted_by_arrival = sort_by_arrival_time(flight_records)
    for record in sorted_by_arrival:
        print(record)
```

Output:

```
Searching for flight ID 602:
Record found: (602, 'Airline B', '11:00 AM', '1:00 PM', 'Delayed')

Sorting by departure time:
(601, 'Airline A', '10:00 AM', '12:00 PM', 'On Time')
(602, 'Airline B', '11:00 AM', '1:00 PM', 'Delayed')
(604, 'Airline D', '1:00 PM', '3:00 PM', 'Cancelled')
(603, 'Airline C', '9:00 AM', '11:00 AM', 'On Time')

Sorting by arrival time:
(603, 'Airline C', '9:00 AM', '11:00 AM', 'On Time')
(601, 'Airline A', '10:00 AM', '12:00 PM', 'On Time')
(602, 'Airline B', '11:00 AM', '1:00 PM', 'Delayed')
(604, 'Airline D', '1:00 PM', '3:00 PM', 'Cancelled')
PS C:\Users\91939\Desktop\3-2\AI Assisted coding>
```

Explanation:

Searching helps farmers quickly access crop details. Sorting crops by moisture level or yield helps in monitoring and decision-making.